DATA 298A Workbook 1

Report Sections 1 and 2

Item	Entry
Team, section and topic	Team 6, Section 22, Topic 12: Neurosymbolic Verification of LLM-Generated Clinical Discharge and Treatment Plans
Date	October 7, 2026

Team members and the sections each person wrote

Team member	Sections written
Sneha	1.1 (shared), 1.2, 1.3, 2.2.1, 2.2.6
Ritika	1.1 (shared), 1.4, 2.2.5, 2.3
Rajesh	1.1 (shared), 1.5, 2.2.2
Akash	1.1 (shared), 2.1, 2.2.4, 2.2.7
Shruthi	1.1 (shared), 2.2.3, 2.2.8, 2.4

1 Introduction

1.1 Project Background and Executive Summary

Authors: Sneha, Ritika, Rajesh, Akash, Shruthi

Our project focuses on improving the safety of LLM-generated clinical discharge and treatment plans. Large language models can generate useful medical text, but they may also produce plans that violate important constraints or fail to recognize their own mistakes. Our goal is to develop a formal verification system that checks an LLM-generated plan against a rule set created by the team. Instead of assuming that a fluent response is safe, the system will classify the plan as valid or invalid and identify the violated rule when a problem is found. This builds on previous work showing limitations in LLM planning and self-verification, including Hager et al. (2024) and Valmeekam et al. (2023a, 2023b).

The project will use MIMIC-IV v3.1 and Ext-CDM v1.1 as its main data sources. Each team member who works with MIMIC-IV will use their own approved copy, and patient data will remain outside the project repository. Our approach combines an LLM-based plan generator with team-defined clinical rules and a formal verifier. During 298A, we will build the rule set, verifier, evaluation harness, benchmark, and Model A, as well as reproduce the selected Hager et al. baseline. Fixed dataset splits and experiment configurations will be used so that results can be reproduced and compared consistently.

The main goal for 298A is to establish a reproducible verification pipeline and demonstrate that it can detect rule violations in generated plans. We target a validity rate of at least 0.85 and an escape rate of no more than 0.05 on a held-out set of 100 cases. In 298B, the project will expand to Models B, C, and D, followed by a four-model comparison, additional ablation and failure analysis, and a software-pipeline case study. The expected contribution is a reusable safety layer that separates plan generation from rule-based verification and shows which verification ideas can transfer beyond the clinical setting. A possible application is a pre-review gate for clinical platforms that draft discharge or treatment plans, where rule violations are flagged before clinician review. A key limitation is that validity means compliance with our formal rules; it does not prove that a plan is clinically correct.

1.2 Project Requirements

Author: Sneha

The system requirements focus on verification, reproducibility, privacy, and measurable performance. The verifier will check LLM-generated discharge or treatment plans against rules defined by the team and provide traceable results when a violation is detected.

ID	Requirement	Metric	Planned Evidence
R1	The system shall classify an LLM-generated plan as valid or invalid using the team-defined rule set.	Validity >= 0.85 on 100 held-out cases	Evaluation results
R2	The verifier shall return the violated rule ID for every detected invalid plan.	Rule ID included with detected violations	Verifier logs
R3	Rules shall be defined and maintained by the team rather than generated by the LLM.	Versioned rule catalog	Rule catalog
R4	The evaluation harness shall run with one command on fixed splits and record the configuration and random seed.	Reproducible harness execution	Run logs
R5	The evaluation shall use fixed, versioned data splits with a 100-case held-out set.	Same held-out set across evaluations	Split files and manifest
R6	The system shall achieve an escape rate of no more than 0.05.	Escape rate <= 0.05	Benchmark results
R7	No patient data shall be committed to the repository or sent to hosted LLM APIs.	No patient data in repository/API workflow	Repository and configuration review
R8	The project shall be reproducible from a clean clone.	Successful clean-clone setup and run	Reproduction test
R9	Each team member who uses MIMIC-IV or Ext-CDM shall use their own PhysioNet-approved copy; dataset versions and integrity information shall be recorded.	Approved access for each data user; dataset versions and hashes recorded	Access-status tracker, manifest and verification log
R10	Model A shall generate discharge or treatment plans that run through the common evaluation harness.	Plan validity rate on held-out cases	Model A run logs
R11	The harness shall report the secondary metrics: repair success, translation fidelity and solver cost.	Targets set before Oct. 8	Harness output

The main success criteria are validity >= 0.85 and escape rate <= 0.05 on the 100 held-out cases. The Hager et al. (2024) baseline will be reproduced separately using the Figure 3 result for OASST 70B in the autonomous setting. Its reported 54.9% represents mean diagnostic accuracy, whereas our validity metric measures whether a generated plan satisfies the defined rules. Therefore, the two numbers represent different tasks and will not be treated as equivalent.

1.3 Project Deliverables

Author: Sneha

The project is divided into 298A and 298B. The main 298A deliverables establish the verification and evaluation pipeline, while 298B expands the work to additional models and analyses.

Deliverable	Owner	Format	Due	Done When
Workbook 1	Team (all five)	Workbook	Oct. 7	Submitted
Team Meeting 1	Team (all five)	Meeting/demo	Oct. 8	Progress demonstrated
Data manifest and fixed splits	Akash (manifest), Rajesh (splits)	Files/documentation	Oct. 8	Versions and split IDs recorded

Deliverable	Owner	Format	Due	Done When
Rule set	Sneha	Rule catalog	Oct. 22 (working target)	Rules have IDs and supporting sources
Verifier and harness	Working plan: Sneha + Shruthi (verifier), Rajesh (harness); confirm in Linear	Python software	Nov. 5 (working target)	Runs on fixed splits and reports violations
Model A	Ritika	Model implementation	Nov. 5 (working target)	Runs through common evaluation harness
Hager baseline	Rajesh + Ritika	Benchmark	Before final report	Result reproduced or gap documented
Progress Report 2	Team (all five)	Report	Oct. 22	Submitted
Workbook 2	Team (all five)	Workbook	Nov. 11	Submitted
Team Meeting 2	Team (all five)	Meeting/demo	Nov. 12	Progress demonstrated
Final report	Team (all five)	Report	Dec. 7	Submitted
Final presentation	Team (all five)	Presentation	Dec. 8	Presented
Models B-D and comparison	Team (all five)	Models + analysis	298B	Four models evaluated consistently
Expanded ablation/failure study	Team (all five)	Analysis	298B	Major failures and components analyzed
Software-pipeline case study	Team (all five)	Case study	298B	Transferable and non-transferable rules identified

1.4 Technology and Solution Survey

Author: Ritika

The technology survey focuses on five design decisions: the formal solver, representation of generated plans, generator model, repair mechanism, and possible planner integration. We compared alternatives based on project fit, implementation complexity, cost, scalability, explainability, and technical risk.

Area	Primary Choice	Alternatives	Main Reason	Cost / licence	Main risk
Solver	Z3	cvc5, SAT, Datalog/Prolog, Python rules, model checkers	Supports arithmetic and Boolean constraints and provides unsat cores	Open-source Z3; licence to be recorded in final technology inventory	Encoding effort for clinical rules
Plan format	Structured JSON/schema	Free text parsed into JSON	Easier and more reliable verification	No licence cost	Parsing errors reduce translation fidelity
Generator	Local open-weight LLM	Hosted API, medical-tuned LLM	Avoids sending MIMIC text to hosted APIs	Hardware only; no API cost	Local model may be weaker; GPU needed for large models
Repair	Solver feedback/counterexample	LLM self-reflection	Provides explicit information about failed constraints	Extra model calls per repair round	Feedback quality limits gains

Area	Primary Choice	Alternatives	Main Reason	Cost / licence	Main risk
Planner hybrid	PDDL + classical planner	LLM-only planning	Provides formal planning structure but requires a hand-written domain	Planner-dependent; use an open-source tool and record its licence before adoption	Needs a hand-written domain; clinical fit uncertain

Z3 is currently our primary solver because it supports both arithmetic and Boolean constraints and can provide information such as unsat cores when constraints conflict. Formal solvers have also been applied to clinical rule settings, including work by Hekmatnejad et al. (2019). However, alternatives such as cvc5 and Datalog remain possible if our rule representation changes.

For the plan representation, a fixed schema is preferred over unrestricted free text because it reduces ambiguity between the generated plan and the verifier. If free text must be translated into structured JSON, errors introduced during this step will be measured as part of translation fidelity rather than silently counted as solver errors. The generator will use a local open-weight model because MIMIC text should not be sent to hosted LLM APIs.

For repair, solver feedback is our primary direction, while LLM self-reflection will be considered as an ablation. Valmeekam et al. (2023a) reported stronger results when external validation was used than when an LLM checked itself. A PDDL-based planner is another possible hybrid approach, but it requires a hand-written planning domain and its suitability for clinical discharge planning is still uncertain. These technologies represent planned design choices; they have not yet been experimentally validated in our project.

1.5 Literature Survey of Existing Research

Section lead: Rajesh | Literature contributors: Sneha, Ritika, Rajesh, Akash, Shruthi

We grouped the literature by the question each paper helps us answer: whether an LLM can plan and check itself reliably, whether a symbolic solver or classical planner improves reliability, how repair feedback should be designed, what safety problems appear in clinical LLM use, and how formal methods have already been used for clinical rules. This organization lets us connect prior work directly to Models A-D and to the verifier rather than listing papers one by one.

Planning and self-checking studies give the main motivation for an external verifier. In PlanBench (Valmeekam et al., 2023b), GPT-4 produced valid plans for 206 of 600 Blocksworld instances (34.3%). Valmeekam et al. (2023a) then tested whether an LLM could check and repair its own plans: the base generator solved 40/100 cases, LLM-based checking reached 55/100, while feedback from the sound VAL verifier reached 88/100. The LLM checker also accepted 38 of 45 invalid plans. Huang et al. (2024) similarly found that self-correction without reliable external feedback can reduce reasoning accuracy. Together, these results support keeping verification outside the generator.

Symbolic and planning hybrids show how that external check can be implemented. Logic-LM (Pan et al., 2023) separates language understanding from symbolic reasoning and reported an average 39.2% gain over standard prompting, but it also shows that translation into the formal representation can fail. SatLM (Ye et al., 2023) uses declarative constraints with a solver, which is close to our choice of Z3. LLM+P (Liu et al., 2023) and Guan et al. (2023) use PDDL and classical planners, but both depend on having a correct domain model. These papers motivate our solver and planner variants while also making translation fidelity and domain-model quality explicit evaluation concerns.

Repair studies add another caution. Olausson et al. (2024) found that self-repair gains can become modest once the cost of repeated model calls is considered, and that feedback quality is a major bottleneck. Reflexion (Shinn et al., 2023) shows that verbal feedback can improve an agent across trials, giving us a useful comparison for solver-generated feedback. In the clinical literature, Singhal et al. (2023) show that strong medical-question performance does not remove safety concerns, while Hager et al. (2024) show limitations in clinical decision making on MIMIC-CDM. Hekmatnejad et al. (2019) demonstrate that SMT and Z3 can check clinical knowledge artifacts. The gap we target is between these areas: we have not found prior work in our reviewed set that formally checks an LLM-written clinical discharge or treatment plan against a team-defined rule set and returns the violated rule for repair.

Baseline for reproduction: Hager et al. (2024), Figure 3, OASST 70B in the autonomous setting, reports 54.9% mean diagnostic accuracy on MIMIC-CDM. We use this only as a separate diagnostic baseline; it is not one of Models A-D and it is not directly comparable with our rule-validity target. The public implementation is our reproduction reference, and any model, hardware, or environment differences will be reported.

Baseline to reproduce

Baseline paper	**Reported number**	**Table / figure**	**Benchmark**	**Code obtained?**
Hager et al. (2024), Nature Medicine 30:2613-2622	54.9% mean diagnostic accuracy (OASST 70B, autonomous setting)	Figure 3	MIMIC-CDM / MIMIC-IV-Ext-CDM	Yes (public repositories; Framework repo cloned and pinned)

Literature matrix

Citation	Problem	Method	Dataset	Results	Relevance to our project
Valmeekam et al. (2023b), PlanBench	Test whether general LLMs can produce valid action plans.	Planning benchmark built from standard IPC-style domains.	Blocksworld and Logistics	GPT-4 generated 206/600 valid Blocksworld plans (34.3%).	Shows why Model A should not be trusted without an external checker.
Valmeekam et al. (2023a), Self-Critique	Test whether an LLM can reliably judge and repair its own plans.	GPT-4 plans were checked by either GPT-4 or the sound VAL verifier.	100 Blocksworld instances	40/100 base; 55/100 with LLM checking; 88/100 with VAL. The LLM approved 38/45 invalid plans.	Direct evidence for using a symbolic verifier instead of an LLM judge.
Huang et al. (2024)	Ask whether reasoning improves when a model critiques itself without external truth signals.	Repeated intrinsic self-correction compared with settings that have oracle feedback.	GSM8K, CommonSense QA, HotpotQA	Without oracle feedback, repeated correction often reduced accuracy.	Supports solver-generated feedback for Model C rather than pure self-reflection.
Kambhampati et al. (2024), LLM-Modulo	Define a role for LLMs when they cannot reliably plan or verify alone.	LLM proposes candidates while external model-based critics check them and return feedback.	Position/framework paper	Conceptual framework rather than a new benchmark result.	Closest architectural idea to our generator-verifier-repair loop; our setting is clinical and uses team-written rules.

Citation	Problem	Method	Dataset	Results	Relevance to our project
Pan et al. (2023), Logic-LM	Reduce errors from doing logical reasoning entirely inside the LLM.	LLM translates a problem into symbolic form; a deterministic solver reasons; solver errors can trigger refinement.	ProofWriter, PrOntoQA, FOLIO, LogicalDeduction, AR-LSAT	Reported +39.2% over standard prompting and +18.4% over CoT on average.	Motivates Model B and makes translation fidelity an explicit metric.
Ye et al. (2023), SatLM	Improve reasoning by expressing a problem as constraints rather than an imperative program.	LLM writes a declarative specification that is solved with SAT/SMT tools including Z3.	GSM-style problems, LSAT, BoardgameQA	Reported +23% over program-aided LMs on GSM-Sys and strong LSAT/BoardgameQA results.	Supports declarative clinical constraints and Z3, while reminding us that correctness depends on the specification.
Liu et al. (2023), LLM+P	Combine natural-language understanding with a sound classical planner.	LLM converts the problem to PDDL; a planner solves it; the result is translated back.	Planning benchmark problems	Reported feasible/optimal planning improvements over LLM-only approaches.	Provides the basic design pattern for Model D and requires a hand-authored domain model.
Guan et al. (2023)	Study whether LLMs can construct formal world/domain models for planning.	LLM drafts PDDL; validators and human feedback correct the model before planning.	Two IPC domains and a household domain	More than 40 actions were modeled and corrected models solved 48 tasks.	Shows that LLM-created formal models need validation; supports our decision to control the clinical rules/domain.
Olausson et al. (2024)	Measure whether repeated self-repair is worth its additional inference cost.	Generate, critique, and repair code while accounting for repeated model calls.	HumanEval and APPS	Repair gains were often modest after cost was considered; feedback quality was a key bottleneck.	Motivates limiting repair rounds and measuring repair cost as well as success.
Shinn et al. (2023), Reflexion	Improve an agent through feedback without updating model weights.	The agent stores verbal reflections from previous trials and uses them in later attempts.	HumanEval, ALFWorld, HotpotQA	Authors report 91% pass@1 on HumanEval and improvements on the other tasks.	Provides a verbal-feedback alternative for the repair ablation against solver counterexamples.
Hager et al. (2024)	Evaluate how LLMs perform in realistic clinical decision-making workflows.	Models interact with a simulated information-gathering framework built from patient cases.	MIMIC-CDM, 2,400 cases across four abdominal conditions	OASST 70B reports 54.9% mean autonomous diagnostic accuracy in Fig. 3.	Provides our separate reproducibility baseline and the clinical motivation for stronger verification.
Singhal et al. (2023)	Evaluate medical knowledge and safety-related qualities of large language models.	MultiMedQA evaluation plus clinician/human assessment; Med-PaLM uses instruction prompt tuning.	MultiMedQA	Flan-PaLM reached 67.6% on MedQA, while human evaluation still identified gaps relative to clinicians.	Shows that high medical QA performance does not by itself establish safe clinical use.
Hekmatnejad et al. (2019)	Check clinical decision-support knowledge artifacts for logical inconsistencies.	Clinical logic is translated to SMT and checked with Z3.	Seven public HL7 knowledge artifacts	Solve times were about 105-180 ms; the checker exposed a contradictory artifact.	Closest formal-method precedent: they verify guideline logic, while we verify an LLM-generated plan against rules.

2 Data and Project Management Plan

2.1 Data Management Plan

Author: Akash

The project uses MIMIC-IV v3.1 as the primary clinical data source and MIMIC-IV-Ext-CDM v1.1 for reproducing the Hager baseline. Each team member who works with restricted data must have individual PhysioNet approval and maintain an authorized copy. At present, Akash and Shruthi have approved access and maintain their own copies of the required datasets. Team members without approved access will work only with synthetic examples, aggregate results, code, documentation, manifests, and other non-sensitive derived materials.

The primary working set for the notebook-based cleaning, preprocessing, and exploratory analysis is the MIMIC-IV-Ext-CDM v1.1 dataset. A separate 17 MB MIMIC-IV extract is also used for notebook development and preprocessing tests. This extract is a limited working copy and is not treated as the complete MIMIC-IV dataset. Results produced from the 17 MB extract will be labeled as preprocessing or development results and will not be presented as full-dataset benchmark results. The complete MIMIC-IV v3.1 data remain available only to authorized users for the project's restricted-data workflow.

Patient data will remain on the authorized user's local machine and outside the GitHub repository. Raw datasets will not be shared, copied, or distributed to unauthorized team members. MIMIC-IV text will not be uploaded to hosted LLM APIs or stored in unauthorized shared services. The repository will contain only code, configuration files, split identifiers, dataset manifests, synthetic examples, and aggregate results. Dataset integrity will be managed using version records and SHA256 manifests. The MIMIC-IV v3.1 download is complete and verified for all 33 of 33 files. The MIMIC-IV-Ext-CDM v1.1 copy has also been downloaded and integrity-checked. Fixed split identifiers will be stored separately from the raw data so that experiments remain reproducible without placing restricted data in the repository.

The team will follow the PhysioNet Credentialed Health Data License, maintain the required training and approvals, and follow PhysioNet's guidance for credentialed data and LLM use. This access-controlled workflow defines how authorized users handle restricted data while allowing the team to share code, manifests, synthetic examples, and aggregate results safely.

2.2 Project Management Plan

2.2.1 Business Problem Framing and Stakeholder Analysis

Author: Sneha

The main decision supported by the system is whether an LLM-generated plan should be released for review, held because it violates a rule, or returned for repair. The verifier is therefore intended to provide an additional safety and traceability layer rather than replace clinical judgment.

The main stakeholders include clinical platform teams that could integrate the verification process, clinicians who would review generated plans, and patients whose safety could be affected by incorrect outputs. For the academic project, the instructor and ISA are also important stakeholders because their feedback determines whether the system meets the course and project requirements.

At this stage, no clinician has been formally consulted as part of the project. Therefore, we will not claim that our rule set has been clinically validated. Where possible, rules will be connected to published clinical guidelines or other reliable sources, and clinical review remains an important future step.

2.2.2 Business Requirements and Success Criteria

Author: Rajesh

The primary quantitative success criteria are a validity rate of at least 0.85 and an escape rate of no more than 0.05 on the fixed 100-case held-out evaluation set. These metrics will be calculated through the evaluation harness using fixed dataset splits so that results remain comparable between experiments.

From a practical perspective, the system should reduce the number of rule-violating plans that pass through verification, provide a rule ID for each detected rejection, and record runtime per plan. This makes the output both measurable and traceable instead of providing only a valid/invalid label.

Secondary metrics will include repair success, translation fidelity, and solver cost. Targets for these secondary measures will be finalized before the October 8 checkpoint so that success criteria are not changed after experimental results are observed.

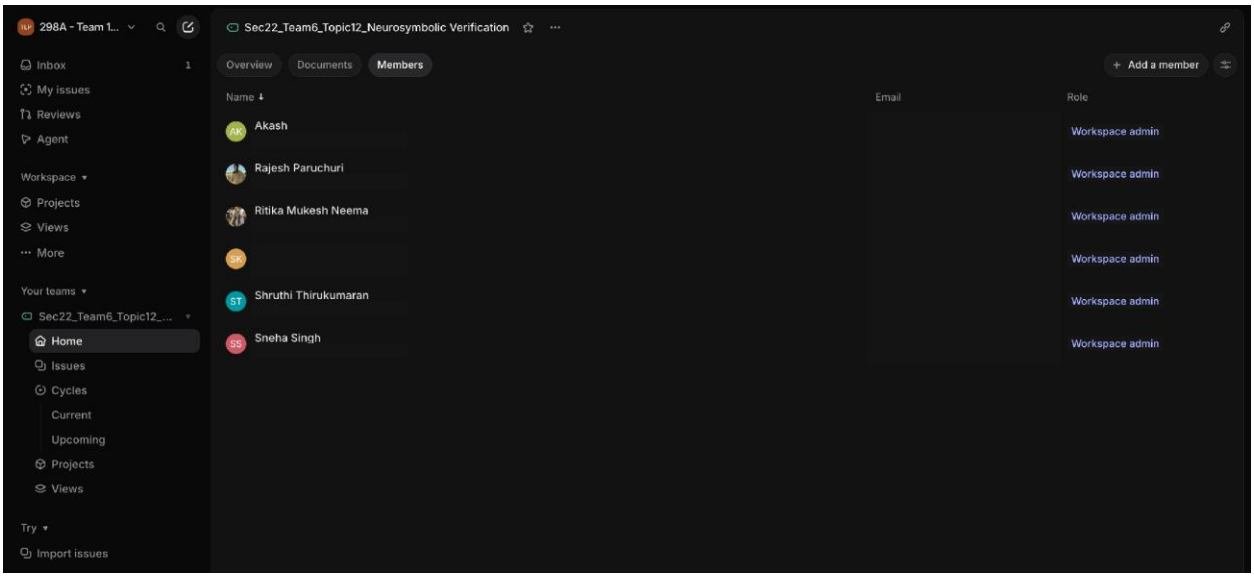
2.2.3 Project Development Methodology

Author: Shruthi

The project follows an iterative, checkpoint-driven development process. During 298A, the team will build the rule set, verifier, evaluation harness, benchmark, Model A, and baseline reproduction. These components provide a common foundation before Models B, C, and D are introduced during 298B.

Course checkpoints are used to review progress and adjust implementation decisions. Major feedback points include the progress reports and Team Meetings on October 8 and November 12. Feedback from these checkpoints will be converted into actionable tasks rather than remaining only in meeting notes.

Linear is organized as one team, **298A Team 6 Neurosymbolic**, with the instructor and ISA invited. The team uses two-week development cycles to organize project work. Each issue has one named primary assignee responsible for moving the task forward, while additional contributors may be documented in the issue description or comments. Every closed issue is required to include a linked artifact, such as a commit, document, experiment result, meeting record, or other evidence of completion. This setup provides traceability from project requirements and feedback to completed work.



2.2.4 Project Organization Plan

Author: Akash

The project is divided into work packages so that major responsibilities have clear ownership. Linear will be used to track these packages and record decisions, progress, and dependencies.

Linear Link: https://linear.app/298a-team-6-neurosymbolic/team/T6/overview

Work package	Owner	Deliverable	Linear project / issue
Data and manifest	Akash	Verified datasets, manifest and documentation	Sec22_Team6_Topic12_Neurosymbolic Verification / **T6-8**
Model A specification	Ritika	Model A design/specification	Sec22_Team6_Topic12_Neurosymbolic Verification / **T6-25**, **T6-26**, **T6-27**
Rule catalog and synthetic cases	Sneha	Versioned rules and test cases	Sec22_Team6_Topic12_Neurosymbolic Verification / **T6-15**, **T6-16**, **T6-19**
Metrics, splits and baseline protocol	Rajesh	Evaluation protocol	Sec22_Team6_Topic12_Neurosymbolic Verification / **T6-9**, **T6-10**, **T6-11**, **T6-12**, **T6-13**, **T6-14**, **T6-30**,
Environment and CI	Shruthi	Reproducible environment	Sec22_Team6_Topic12_Neurosymbolic Verification / **T6-7**, **T6-42**
Verifier and harness	Shruthi (primary owner), Sneha (verifier), Rajesh (harness)	Working verification pipeline	Sec22_Team6_Topic12_Neurosymbolic Verification / **T6-17**, **T6-18**, **T6-20**, **T6-21**, **T6-22**, **T6-23**, **T6-24**, **T6-31**

Each work package will have one primary owner even when multiple team members contribute. This makes it clear who is responsible for moving a task forward and providing evidence that it has been completed.

2.2.5 Scope Management and Prioritization Plan

Author: Ritika

The 298A scope includes the single hospital-encounter setting, formal verifier, rule set, evaluation harness, benchmark, Model A, and Hager baseline reproduction. These components are necessary to establish whether the verification approach can be implemented and evaluated reproducibly.

Models B, C, and D are outside the 298A scope and will be developed in 298B. Full clinical decision support and production deployment are also outside the current project scope. The system is intended as an experimental verification framework rather than a deployed medical product.

Requirements will be prioritized as must-have, should-have, and nice-to-have. If schedule or compute limitations require trade-offs, the team will prioritize the requirements needed to preserve the defined validity and escape-rate criteria and maintain reproducibility.

2.2.6 Risk, Issue, and Change Management

Author: Sneha

The project risks will be tracked using explicit triggers so that the team can respond before they significantly affect project milestones. The highest risks currently involve data access, compute availability, software compatibility, metric interpretation, and clinical support for the rules.

Risk	**Likelihood / impact**	**Trigger**	**Mitigation**	**Owner**
Delayed MIMIC access	Medium / High	Member lacks access when data work begins	Authorized users maintain individual copies; members without access use synthetic and aggregate materials until approval	Akash
Slow MIMIC download	Medium / Medium	Required MIMIC tables are not available on schedule	Prioritize required tables and use Ext-CDM where appropriate	Akash
MIMIC data integrity issue	Low / High	A file is missing or fails SHA256 verification	Re-download the affected file, compare it with the manifest, and record the verification result	Akash

Risk	Likelihood / impact	Trigger	Mitigation	Owner
No GPU for 70B baseline	High / High	Suitable GPU unavailable by agreed checkpoint	Seek university GPU/HPC; otherwise use smaller model/subset and document gap	Ritika
Hager environment incompatibility	Medium / High	Repository fails during environment setup	Pin versions and test the CUDA stack early	Shruthi
Accuracy and validity are confused	Medium / High	Results compare the metrics directly	Maintain task/metric mapping and confirm with instructor	Rajesh
Rules lack clinical validation	Medium / High	Rule has no supporting clinical source	Cite guidelines and request review when possible	Sneha

The observed MIMIC download rate of approximately 60-100 KB/s came from one team member's machine on October 1 and is therefore treated as a project observation rather than a general download rate. Evaluation metrics will be frozen by October 8.

2.2.7 Communication, Review, and Feedback Plan

Author: Akash

The team will hold a weekly synchronization meeting to discuss progress, blockers, dependencies, and upcoming deadlines. Important decisions will be recorded in the Linear project so that changes can be traced rather than relying on informal conversations.

Feedback from the instructor and ISA will be converted into Linear issues whenever it requires an action. Each issue will have a named primary assignee, an appropriate cycle, and a linked artifact or evidence before it is closed. Artifacts may include a code commit, document, dataset manifest, experiment result, meeting record, screenshot, or test output.

Meeting minutes will record decisions, assigned actions, unresolved questions, and links to the corresponding Linear issues. This process will provide evidence of how the team communicated, responded to feedback, and completed project work.

2.2.8 Applied Project Management for Business Problem Solving

Author: Shruthi

Project management practices will be demonstrated using actual project decisions rather than hypothetical examples. One early example was the discovery that credentialed MIMIC access could not simply be shared across the team. The team responded by requiring each member who needs the dataset to obtain and download their own approved copy.

A second example is the distinction between Hager's diagnostic accuracy and our rule-validity metric. Identifying this difference early prevents the project from comparing two numbers that measure different tasks. The team also identified the hardware requirements of the Hager baseline before the final benchmarking stage, allowing time to investigate university GPU resources or document an alternative setup.

Evidence for these decisions is recorded through the Linear project, issue IDs, meeting notes, repository commits, dataset manifests, and other linked project artifacts. These artifacts provide traceability from identified risks and project decisions to the actions taken and the resulting evidence of completion.

2.3 Project Resource Requirements and Plan

Author: Ritika

The project currently has access to an Apple M4 laptop with 16 GB RAM and approximately 150 GB of measured free storage. Additional team hardware will be added once each member confirms their available resources.

Resource	Status	Purpose	Cost
Apple M4 laptop, 16 GB RAM	Available	Development, rule and verifier work, smaller experiments	Owned
About 150 GB free storage	Available	Local project files and authorized data	Owned
NVIDIA GPU capable of quantized 70B inference	Needed	Exact Hager baseline reproduction	Not yet budgeted
University GPU or HPC	Needed (not yet requested)	Close the compute gap	To be confirmed with university access
Python and Z3	Planned	Implementation and formal verification	Free
pytest	Planned	Testing	Free
GitHub	Available	Version control	No incremental project cost identified
Linear	Available	Project management	No incremental project cost identified
Experiment tracker	To be selected	Experiment configuration and results	Choose a no-cost/local option if possible
MIMIC-IV v3.1	Downloaded and verified: 33 of 33 files; SHA256 verification complete	Clinical dataset	Free
Ext-CDM v1.1	Downloaded and verified: v1.1, 13 files, ~17 MB, 2,400 cases; 12/12 SHA256-listed files match.	Baseline and data experiments	Free

The largest compute gap is the exact reproduction of the 70B baseline, which cannot reasonably run on the currently identified 16 GB M4 laptop. The preferred solution is university GPU or HPC access. If this is unavailable, a smaller model or subset may be used, but the difference from the original baseline will be clearly documented. Hosted LLM APIs will not be used with MIMIC text.

2.4 Project Schedule

Author: Shruthi

The project schedule follows the official course deadlines and divides implementation work into short development cycles. Dependencies are tracked so that work required by later experiments is completed first.

Milestone	Date	Owner	Depends on	Status
Workbook 1	Oct. 7	Team	Sections 1-2 drafted	Complete
Team Meeting 1	Oct. 8	Team	Fixed splits, metrics, data workflow	Planned
Progress Report 2	Oct. 22	Team	Status evidence from Linear	Planned
Workbook 2	Nov. 11	Team	Sections 3-4	Planned
Team Meeting 2	Nov. 12	Team	Model A running, harness	Planned
Final Report	Dec. 7	Team	All sections	Planned
Final Presentation	Dec. 8	Team	Final report	Planned

The completed/planned cycles are September 10-24, September 25-October 8, October 9-22, October 23-November 5, November 6-12, November 13-26, and November 27-December 7. The main dependency path is data access -> manifest -> fixed splits -> harness -> Model A -> benchmark, while rules -> verifier -> harness forms a second path and GPU access -> baseline reproduction forms another. A three- to four-day buffer will be maintained before major submissions where possible. If the schedule chart and active project board differ, the current Linear board will be treated as the operational source of truth.

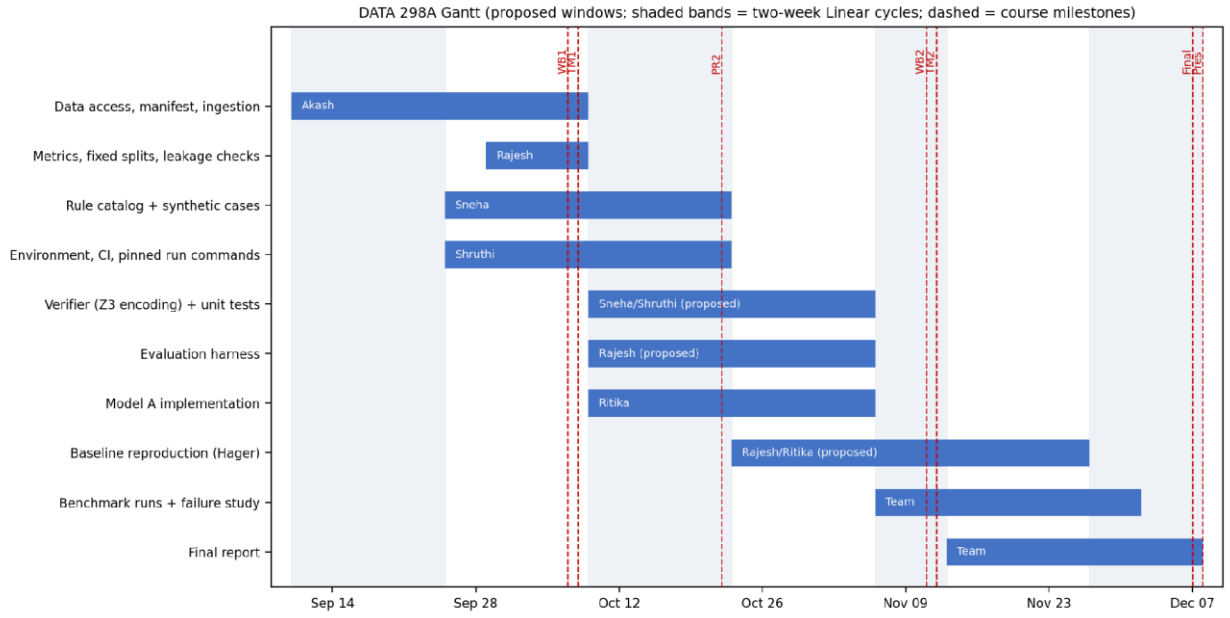


Figure 2.4-1. Gantt chart (proposed task windows; shaded bands are the two-week Linear cycles). Produced by scripts/build_schedule_charts.py..

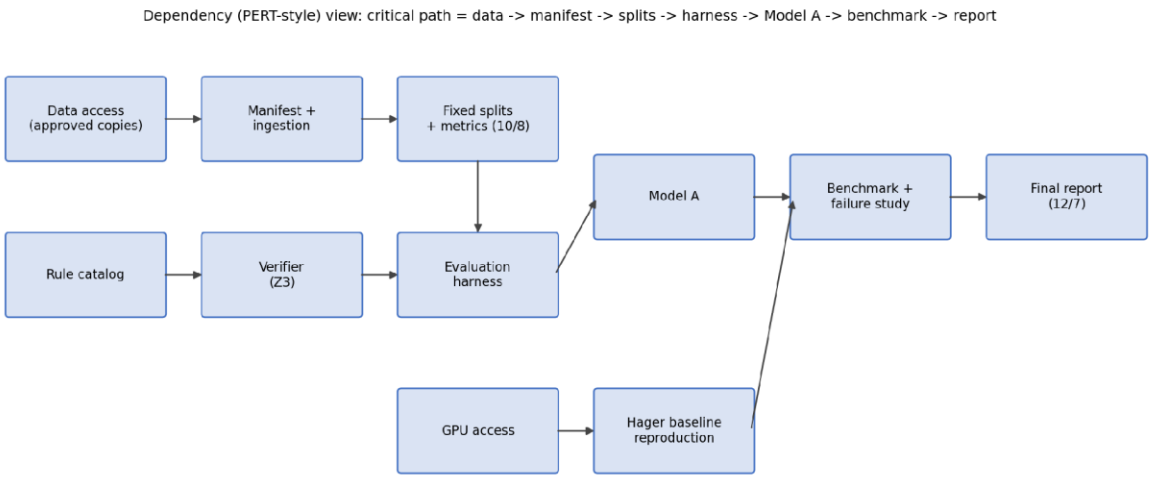


Figure 2.4-2. Dependency (PERT-style) view. Produced by project/scripts/build_schedule_charts.py.

References

[1] Guan, L., Valmeekam, K., Sreedharan, S., & Kambhampati, S. (2023). Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. Advances in Neural Information Processing Systems, 36. https://arxiv.org/abs/2305.14909

[2] Hager, P., Jungmann, F., Holland, R., Bhagat, K., Hubrecht, I., Knauer, M., Vielhauer, J., Makowski, M., Braren, R., Kaissis, G., & Rueckert, D. (2024). Evaluation and mitigation of the limitations of large language models in clinical decision-making. Nature Medicine, 30, 2613-2622. https://doi.org/10.1038/s41591-024-03097-1

[3] Hekmatnejad, M., Simms, A. M., & Fainekos, G. (2019). Model Checking Clinical Decision Support Systems Using SMT. arXiv:1901.04545. https://arxiv.org/abs/1901.04545

[4] Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). Large language models cannot self-correct reasoning yet. International Conference on Learning Representations. https://arxiv.org/abs/2310.01798

[5] Kambhampati, S., Valmeekam, K., Guan, L., Verma, M., Stechly, K., Bhambri, S., Saldyt, L. P., & Murthy, A. B. (2024). Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks. Proceedings of the 41st International Conference on Machine Learning, PMLR 235, 22895-22907. https://proceedings.mlr.press/v235/kambhampati24a.html

[6] Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., & Stone, P. (2023). LLM+P: Empowering large language models with optimal planning proficiency. arXiv:2304.11477. https://arxiv.org/abs/2304.11477

[7] Olausson, T. X., Inala, J. P., Wang, C., Gao, J., & Solar-Lezama, A. (2024). Is self-repair a silver bullet for code generation? International Conference on Learning Representations. https://arxiv.org/abs/2306.09896

[8] Pan, L., Albalak, A., Wang, X., & Wang, W. Y. (2023). Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. Findings of the Association for Computational Linguistics: EMNLP 2023, 3806-3824. https://doi.org/10.18653/v1/2023.findings-emnlp.248

[9] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36. https://arxiv.org/abs/2303.11366

[10] Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., et al. (2023). Large language models encode clinical knowledge. Nature, 620, 172-180. https://doi.org/10.1038/s41586-023-06291-2

[11] Valmeekam, K., Marquez, M., & Kambhampati, S. (2023a). Can large language models really improve by self-critiquing their own plans? arXiv:2310.08118. https://arxiv.org/abs/2310.08118

[12] Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S., & Kambhampati, S. (2023b). PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. Advances in Neural Information Processing Systems, 36 (Datasets and Benchmarks). https://arxiv.org/abs/2206.10498

[13] Ye, X., Chen, Q., Dillig, I., & Durrett, G. (2023). SatLM: Satisfiability-aided language models using declarative prompting. Advances in Neural Information Processing Systems, 36. https://arxiv.org/abs/2305.09656

Submission checklist (not part of the report)

Check	Yes / No
Every section above is written as report prose, not as notes	Yes
Every claim links to a commit, run, dataset version or Linear issue	Yes
Every figure names the script or notebook that produced it	Yes
Every closed Linear issue for this period has a linked artifact	Yes
Each section names the team member who wrote it	Yes

Self-assessment and gap register

Rate honestly (poor / good / excellent). Any row below good needs an owner and a date. Graded on accuracy, not optimism.

Report section	Our rating	If below good: what is missing	Owner	Lands by
1.1-1.5	Good	Confirm technical owners/dates, finish literature expansion, and verify remaining licence details.	Sneha (1.2, 1.3); Rajesh/Ritika for literature/technology	Oct. 5
References	Good	Verify bibliographic details and keep only sources actually read by the team.	Rajesh + all contributors	Oct. 5